

PenBox — Notice d'utilisation

Interface graphique (PySide6) pour lancer et suivre les ~34 scripts de pentest du dépôt, sans passer par le menu terminal `cybersec_launcher.py`.

1. Installation (une seule fois)

Prérequis : `uv` dans le PATH.

```
setup.bat
```

Ce script crée un venv dédié 64 bits (`.venv-penbox`) et installe les dépendances (`requirements.txt` , dont `PySide6` , `PyYAML` , `paramiko` , `cryptography` , `psutil`). Un venv dédié est nécessaire car PySide6 n'a pas de build 32 bits, et le Python système peut l'être.

2. Lancer l'app

Double-clic sur `penbox.bat` à la racine du dépôt. Il vérifie que `.venv-penbox` existe (sinon il demande de lancer `setup.bat` d'abord) puis démarre `penbox_app.py`.

Au tout premier lancement, un avertissement légal s'affiche une fois (mémorisé par utilisateur Windows, pas par projet). Refuser ferme l'app sans ouvrir la fenêtre principale.

3. Vue d'ensemble de l'interface

- **Panneau "Cibles" (gauche)** — sélecteur de projet, ajout/suppression de cibles (IP, domaine, URL, fichier, dossier...), liste à cases à cocher. Bouton **"Récupérer via SFTP..."** pour rapatrier des fichiers distants (voir §6).
- **Le menu déroulant à côté du champ de saisie de la cible** sert à indiquer le **type** de la valeur que tu ajoutes : `host` , `ip` , `domain` , `url` , `cidr` , `file` , `dir` ou `username` . Il a deux effets concrets :
 1. **Validation à l'ajout** — la valeur est vérifiée selon le format attendu par le type choisi (ex: `ip` refuse `exemple.com` , `url` exige `http://` ou `https://` , `cidr` exige une notation type `192.168.1.0/24`). Si le type ne correspond pas à la valeur saisie, l'ajout est refusé avec un message d'erreur.
 2. **Filtrage des scripts compatibles** — au lancement, PenBox n'associe un script qu'aux cibles cochées dont le type correspond à ce que ce script attend (un script prévu pour `host` accepte aussi `ip` et `domain`). Une cible du mauvais type est silencieusement ignorée pour ce script ("aucune cible cochée de type ...") — donc si un script que tu attendais de voir tourner n'apparaît pas dans la Console, vérifier en premier le type choisi pour la cible. - Sélectionner `file` ou `dir` fait apparaître un bouton "Parcourir..." pour choisir le chemin via l'explorateur plutôt que de le taper à la main.

- **Onglet "Historique" (gauche, sous le même dock que Cibles)** — liste des runs passés du projet, persistés en base même après un redémarrage de PenBox (voir §9).
- **Panneau "Catalogue" (centre)** — arbre des scripts groupés par catégorie (recon, network, vulnassess, logs, crypto, remediation, exploit, privesc, web, osint, forensic), case à cocher par script, filtre de recherche, description au survol/sélection. Sélecteur de **Playbook** en haut pour charger/enregistrer/supprimer une sélection de scripts nommée (voir §5). Boutons **"Lancer la sélection"** et "🌐 **Exécuter à distance (SSH)...**".
- **Panneau "Console" (bas)** — un onglet par job lancé, stdout/stderr séparés et colorés, bouton Kill, barre de progression globale, icône de statut par job (● en cours / ✅ ok / ❌ erreur, timeout ou tué). Barre de recherche sous les onglets (texte ou regex, surlignage, navigation ◀ ▶, compteur de correspondances) — cherche uniquement dans l'onglet actuellement affiché.
- **Panneau "Résultats" (centre droit)** — table filtrable des findings (texte, niveau de risque, script d'origine, case "Masquer les faux-positifs"), avec un détail par ligne sélectionnée (champs + arbre JSON brut). **Clic droit sur une ligne** pour la marquer faux-positif ou changer son niveau de risque (voir §10).
- **Export** — HTML (rapport avec badges colorés par risque) ou CSV, depuis le panneau Résultats.
- **Menu "Paramètres"** (barre de menu, en haut) — mode SSH strict/tolérant, gestion des clés hôtes SSH, profils de connexion (coffre-fort), chaînage automatique (voir §7 à §11).
- **Barre de statut (bas de la fenêtre)** — indicateur CPU / RAM / nombre de jobs actifs, pour calibrer la parallélisation si la machine sature.

4. Workflow type

1. Ajouter une ou plusieurs cibles dans le panneau Cibles, les cocher.
2. Cocher un ou plusieurs scripts dans le Catalogue (ou charger un Playbook, voir §5).
3. "Lancer la sélection" — PenBox associe chaque script coché à chaque cible cochée compatible avec son mode d'entrée (`target` , `multi_target` , `file_input` , `payload` , `none`). Pour les modes non liés à une cible directe, un formulaire (`ParamFormDialog`) s'ouvre pour saisir les paramètres et, si besoin, surcharger le timeout.
4. Suivre l'avancement dans la Console (3 jobs en parallèle par défaut).
5. Consulter/filtrer les résultats, exporter si besoin, ou revenir dessus plus tard via l'onglet Historique (§9).

5. Playbooks (scénarios de scripts sauvegardés)

En haut du panneau Catalogue : sélecteur **"— Playbook —"** + boutons.

- **"Enregistrer la sélection..."** — sauvegarde les scripts actuellement cochés sous un nom (ex. "Recon Externe Rapide"). Écrase un playbook existant du même nom après confirmation.
- **Charger** — coche automatiquement les scripts du playbook sélectionné dans le combo (décoche le reste). Si le catalogue a changé depuis l'enregistrement (script renommé/supprimé), les scripts manquants sont signalés et ignorés plutôt que de faire échouer le chargement.
- **Supprimer** — retire le playbook sélectionné.

Les playbooks sont partagés entre tous les projets (stockés au niveau de la base `penbox.db`, pas par projet).

6. Récupération de fichiers via SFTP

Certains scripts travaillent uniquement sur le filesystem local (`env_secrets_scanner`, `log_parser`, `metadata_extractor`, `timeline_builder`, etc.) et ne savent pas aller chercher un fichier sur une machine distante eux-mêmes. Le bouton **"Récupérer via SFTP..."** du panneau Cibles ouvre une boîte de dialogue :

- **"Charger un profil..." / "Enregistrer comme profil..."** — voir §8 (coffre-fort) pour réutiliser des identifiants fréquents sans les ressaisir.
- Hôte, Port (22 par défaut), Utilisateur
- Authentification par mot de passe ou par clé privée (parcourir un fichier)
- Chemin distant (fichier ou dossier — le téléchargement est récursif pour un dossier)
- Nom local (proposé automatiquement, modifiable)

Le téléchargement se fait en arrière-plan (l'UI ne se fige pas) vers `.penbox_fetched/<nom>/` à la racine du dépôt. Une fois terminé, le bouton **"Ajouter comme cible"** l'enregistre directement comme cible locale (type fichier ou dossier).

- **Barre de progression** — pour un dossier, PenBox fait d'abord un comptage léger (métadonnées seules) pour connaître le total, puis affiche "X/Y fichiers (nom en cours)" en direct. Le SFTP transfère un fichier à la fois : un gros dossier (`/home/...`, `/etc`, etc.) prend normalement plusieurs dizaines de secondes à quelques minutes selon le nombre de fichiers — c'est attendu, pas un blocage.
- **Fichiers illisibles ignorés, pas de transfert avorté** — un fichier avec permissions refusées, un lien symbolique cassé ou une socket Unix (ex. `~/gnupg/S.gpg-agent`, courant dans un `$HOME`) ne fait plus échouer tout le dossier : il est journalisé et sauté, le reste continue. Le message final liste les fichiers ignorés s'il y en a.
- **Annuler un gros transfert en cours** — cliquer sur "Annuler" (le bouton "Close" se relabellise pendant un transfert) demande un arrêt propre plutôt que de fermer brutalement la fenêtre : le statut passe à "Annulation en cours..." le temps que le transfert s'arrête (vérifié entre chaque fichier, donc quasi immédiat sauf si un fichier précis est en train de se transférer). Fermer la fenêtre en pleine récupération d'un gros dossier ne fait plus planter PenBox.
- **Vérification de la clé d'hôte** — voir §9, même mécanisme que pour l'exécution distante.

7. Exécution à distance via SSH

Certains scripts n'ont de sens que lancés *sur* la machine cible plutôt que sur ta machine locale — `privesc_checker`, `system_hardener`, `artifact_collector`, `patch_manager` (mode `none`, pas de cible). Le bouton "🌐 **Exécuter à distance (SSH)...**" du panneau Catalogue fait ça : il upload le script sélectionné sur la machine distante par SFTP, l'exécute là-bas via SSH, rapatrie le JSON de résultat, et l'intègre au panneau Résultats exactement comme un run local.

- **"Charger un profil..." / "Enregistrer comme profil..."** — mêmes boutons que pour le SFTP (§6), voir §8.
- Fonctionne pour n'importe quel script du catalogue, pas seulement les modes `none` — pour un script `target` (ex. `log_parser -f`, `port_scanner -t`), la valeur cochée dans Cibles est passée telle quelle au script exécuté côté distant (donc un chemin de fichier doit exister *sur la cible*, une IP doit être joignable *depuis la cible*).
- Prérequis côté cible : un interpréteur Python accessible (par défaut `python3`, modifiable dans le dialogue) — tous les scripts du catalogue sont mono-fichier et stdlib-only, aucune dépendance à installer côté cible.
- Mêmes champs de connexion que le fetch SFTP (hôte/port/utilisateur, mot de passe ou clé privée), plus l'interpréteur Python distant.
- **Le Kill dans la Console tue réellement le process distant** : PenBox capture le PID du process au lancement et envoie un `kill -9` explicite dessus (en plus de fermer le canal SSH), pour éviter qu'un process orphelin continue de tourner sur la cible si la simple fermeture du canal n'avait pas suffi.
- Les fichiers uploadés/temporaires sont nettoyés sur la cible (`/tmp/.penbox_remote/...`) à la fin du run.

8. Coffre-fort de profils de connexion SSH

Pour éviter de ressaisir hôte/port/utilisateur/mot de passe à chaque fetch SFTP ou exécution distante :

Paramètres > Profils de connexion SSH..., ou directement les boutons "Charger un profil..." / "Enregistrer comme profil..." dans les dialogues SFTP et SSH.

- **Premier usage** — PenBox demande de créer un **mot de passe maître**. Il chiffre tous les profils enregistrés (Fernet, clé dérivée par PBKDF2HMAC) dans `.penbox_vault.enc` à la racine du dépôt. Ce mot de passe **n'est jamais stocké** : s'il est perdu, les profils enregistrés sont irrécupérables (il faut recréer le coffre).
- **Usages suivants** — le mot de passe maître n'est redemandé qu'une fois par lancement de PenBox (le coffre reste déverrouillé en mémoire pour la session).
- **Gestion** — Paramètres > Profils de connexion SSH... liste tous les profils enregistrés (nom, hôte, utilisateur, méthode d'authentification), avec Ajouter/Modifier/Supprimer.
- `.penbox_vault.enc` est ignoré par git (comme `penbox.db`, `.penbox_known_hosts`) — il ne quitte jamais ta machine.

9. Vérification des clés hôtes SSH (protection MITM)

PenBox mémorise la clé hôte de chaque serveur SSH contacté (fichier `.penbox_known_hosts`, format compatible `known_hosts` OpenSSH) et **refuse toute connexion où la clé a changé depuis la dernière fois** — que ce soit un vrai risque d'interception (MITM) ou simplement une machine réinstallée.

- **Mode Tolérant (par défaut)** — la première connexion à un hôte inconnu enregistre automatiquement sa clé (TOFU, "Trust On First Use") ; un message dans le dialogue (fetch) ou la Console (exécution

distante) indique l'empreinte SHA256 approuvée. Toute connexion *suivante* avec une clé différente pour ce même hôte est refusée, quel que soit le mode.

- **Mode Strict** — activable via **Paramètres > Mode strict SSH** : refuse aussi les hôtes jamais vus, il faut les approuver au préalable.
- **Paramètres > Clés hôtes SSH...** — consulter les empreintes enregistrées, ou en supprimer une si un changement de clé est légitime (ex. cible réinstallée) — la prochaine connexion sera alors traitée comme un premier contact.

10. Historique et comparaison de scans

L'onglet **Historique** (tabé avec Cibles, à gauche) liste tous les runs passés du projet sélectionné — outil, cible, statut, date, nombre de findings — même après un redémarrage de PenBox (les runs sont en base SQLite ; seule la Console vit en mémoire et se vide à la fermeture).

- **"Voir la sortie..."** (ou double-clic) — ouvre le stdout/stderr complet d'un run passé, pour un job dont l'onglet Console d'origine a été fermé.
- **"Filtrer les résultats sur ce run"** / **"Afficher tous les résultats"** — restreint le panneau Résultats à un seul run, pratique pour isoler un scan précis dans un projet qui en a accumulé beaucoup.
- **"Comparer deux scans..."** — choisit deux runs (même outil, deux dates différentes) et liste les findings  **nouveaux** ou  **disparus** entre les deux, pour suivre l'évolution d'une cible dans le temps.

11. Édition des résultats : faux-positifs et niveau de risque

Clic droit sur une ligne du panneau Résultats :

- **Marquer/retirer comme faux-positif** — la ligne apparaît grisée et barrée, et disparaît de la vue tant que la case **"Masquer les faux-positifs"** (au-dessus de la table) reste cochée (décochable pour tout revoir).
- **Changer le niveau de risque** — surcharge manuellement `critical` / `high` / `medium` / `low` / `info`, par exemple pour remonter un finding jugé plus grave dans le contexte métier.

Ces changements sont persistés en base et n'affectent pas la comparaison de scans (\$10), qui compare toujours le contenu brut des findings.

12. Chaînage entre scripts

Après un run réussi de `subdomain_enum` (→ nouveaux domaines) ou `network_mapper` (→ nouvelles IP) avec des résultats :

- **Par défaut** — une fenêtre "Envoyer vers Cibles" s'ouvre pour choisir manuellement lesquelles des découvertes ajouter comme nouvelles cibles.
- **Chaînage automatique** (**Paramètres > Chaînage automatique**, désactivé par défaut) — saute cette confirmation : toutes les découvertes sont ajoutées comme cibles, **et** PenBox relance automatiquement

`port_scanner` / `ssl_checker` (pour des domaines) ou `port_scanner` (pour des IP) dessus, sans intervention. À activer seulement si tu es à l'aise avec des scans qui s'enchaînent tout seuls sur ce que trouve la recon.

13. Particularités à connaître

- **Statut "ok" malgré un code de sortie non nul** — `env_secrets_scanner` et `privesc_checker` sortent en code 1 quand ils trouvent quelque chose (comportement type grep). PenBox affiche quand même  si le JSON a été correctement parsé. Rien ne distingue visuellement ce cas d'un run "propre" dans la Console — c'est normal, pas un bug.
- **Port personnalisé pour `ssl_checker`** — au lancement, une petite fenêtre demande le port à tester (443 par défaut). Annuler revient au port par défaut.
- `memory_dump` — le champ "Valeur" attend un PID numérique **ou** un nom de processus (ex. `4821` ou `notepad.exe`) ; non supporté sur macOS (restrictions SIP). Le fichier produit peut contenir des secrets en clair, à traiter comme sensible.
- **Kill d'un job local qui ne répond pas** — le bouton Kill n'est pas instantané : PenBox retente en `taskkill /F /T` après 5 secondes, puis abandonne le suivi après 20 secondes (message dans l'onglet du job). Dans ce cas, vérifier/tuer manuellement le processus via le Gestionnaire des tâches. Déjà observé avec `whois_lookup.py` qui peut rester bloqué sur un appel réseau. (Pour un run distant, voir §7 — le kill y est désormais direct côté cible.)
- `file_encryptor` / `hash_generator` — le catalogue les verrouille sur un seul sous-mode dans l'UI (chiffrement / hash). Le déchiffrement reste accessible uniquement en ligne de commande.

14. En cas de souci

- App qui ne démarre pas → vérifier que `setup.bat` a bien tourné sans erreur et que `.venv-penbox\Scripts\python.exe` existe.
- Un script individuel plante en dehors de PenBox → il a probablement un vrai bug (comme `env_secrets_scanner.py` avant correctif) ; le reproduire en CLI direct dans `.venv-penbox` pour investiguer.
- Fenêtre/process qui semble disparaître en étant minimisé → connu sur cette machine (comportement ASUS lié au system tray), le process continue de tourner, ce n'est pas un crash.
- Mot de passe maître du coffre-fort oublié → aucune récupération possible (rien n'est stocké en clair) ; supprimer `.penbox_vault.enc` recrée un coffre vide au prochain accès à un profil.